



Activity Sheet

Subject Name: Integrative and Programming Technologies

Date: 03/03/2026

☒ Lecture ☐ Laboratory No.: 1

Course/Year: BSIT - II

Instructor's Name: Mary-Ann Mzana

This code retrieves user input from a form using the `$_POST` superglobal and stores it in the variable `$username`. It then outputs the value directly to the browser by concatenating it with the string "Welcome ". At first glance, this seems harmless. However, because the input comes directly from the user, it introduces several potential problems.

First, if the user enters nothing in the username field and submits the form, the variable `$username` will contain an empty string. As a result, the output will simply display "Welcome " with nothing following it. While this does not immediately cause a crash, it shows that the code lacks proper validation. A well-designed application should check whether required fields are empty before processing them. Additionally, if the form is submitted without the `username` field existing at all, PHP may generate an "Undefined index" warning because the code assumes that `$_POST['username']` is always set.

Second, a more serious issue occurs if the user enters malicious input, such as HTML or JavaScript code. For example, if the user types:

```
HTML
<script>alert('Hacked')</script>
```

The script will output:

```
HTML
Welcome <script>alert('Hacked')</script>
```



Because the code directly prints user input without any filtering or escaping, the browser will interpret the `<script>` tags as executable JavaScript. This results in a Cross-Site Scripting (XSS) vulnerability. XSS is a common web security flaw that allows attackers to inject malicious scripts into webpages viewed by other users. Through XSS, attackers can steal session cookies, hijack user accounts, redirect victims to malicious websites, or manipulate page content.

The root cause of the vulnerability is that the program trusts user input and displays it directly in the HTML response. In web development, all user input must be treated as untrusted data. Data coming from `$_POST`, `$_GET`, or any external source can be manipulated by an attacker. Therefore, outputting raw user input without sanitization makes the application insecure.

The given code is not secure because it lacks input validation and output escaping. While it may function correctly under normal conditions, it is vulnerable to XSS attacks and may also generate warnings if the expected form field is missing. To make the code secure, developers should validate that the input exists and is not empty, and they should escape special characters using functions such as `htmlspecialchars()` before displaying the data in the browser. This ensures that any HTML or JavaScript entered by the user is treated as plain text rather than executable code.



LORMA COLLEGES
CLI Campus
San Juan, Urbiztondo, La Union

